

Types of Variables in Java

Variables in Java are classified based on where they are declared.

There are three types of variables:

1. Local Variable
2. Instance Variable (Non static variable)
3. Static Variable

1. Local Variable

A variable declared inside a method, constructor, or block is called a **Local Variable**.

Characteristics

- Declared inside a method, constructor, or block.
- Accessible only within that method or block.
- Created when the method starts executing.
- Destroyed when the method execution completes.
- Must be initialized before use.

Example

```
public class Student {  
    public static void main(String[] args) {  
        int marks = 90; // Local Variable  
        System.out.println(marks);  
    }  
}
```

Output

90

2. Instance Variable

A variable declared inside a class but outside all methods, constructors, and blocks is called an **Instance Variable**.

Characteristics

- Declared inside a class but outside methods.
- Belongs to an object.
- Every object gets a separate copy.
- Created when an object is created.
- Destroyed when the object is destroyed.
- Gets default values if not initialized.

Example

```
class Student {  
    String name = "Rahul"; // Instance Variable  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        System.out.println(s1.name);  
    }  
}
```

Output

Rahul

3. Static Variable

A variable declared with the static keyword inside a class but outside all methods, constructors, and blocks is called a **Static Variable**.

Characteristics

- Declared using the static keyword.
- Belongs to the class, not to objects.
- Only one copy exists in memory.
- Shared by all objects.
- Created when the class is loaded.
- Can be accessed using the class name.

Example

```
class Student {  
    static String institute = "QspidersJspiders";  
    public static void main(String[] args) {  
        System.out.println(Student.institute);  
        // or  
        System.out.println(institute);  
    }  
}
```

Output

ABC Institute

Feature	Local Variable	Instance Variable	Static Variable
---------	----------------	-------------------	-----------------

Declared Inside	Method	Class	Class
Keyword Required	No	No	static
Belongs To	Method	Object	Class
Number of Copies	Every Method Call	Every Object	Only One
Default Value	No	Yes	Yes
Accessed By	Directly Inside Method	Object	Class Name/Object

Intro to Class and Object Concept:

Class in java

A class is a blueprint or template used to create objects.

It defines the properties (data) and behaviors (methods) that objects will have.

Characteristics of a Class

- A class is a blueprint for creating objects.
- A class can contain variables and methods.
- A class does not store actual data.
- One class can be used to create many objects.

Syntax

```
class ClassName {
```

```
}
```

Example

```
class Student {
```

```
}
```

Here:

- **class** → Keyword used to create a class.
- **Student** → Class Name.

Object in Java

An object is a real-world entity created from a class.

It is an instance of a class that contains its own data and can perform actions defined in the class.

Memory is allocated only when an object is created.

Think of a class as a blueprint of a house.

Class → House Blueprint

Object → Actual House

Similarly:

Class → Template

Object → Real Entity

Syntax

```
ClassName objectName = new ClassName();
```

Characteristics of an Object

- An object is an instance of a class.
- Memory is allocated when an object is created.
- An object can access variables and methods of a class.
- Multiple objects can be created from the same class.
- Each object has its own state (data).

```
Student s1 = new Student(); // this line represents Object creation
```

Student → Class Name

s1 → Object Name (Reference Variable)

new → When we use the new keyword, Java allocates memory for the object

Student() → Constructor call that initializes the object

Note : Class is a blueprint, and Object is the real entity created from that blueprint.